

航空票务系统后端对接文档

本文档面向两类协作对象：前端开发同学与 MySQL 数据库开发同学。它的目标不是描述“系统应该有什么”，而是把后端在联调时需要的接口边界、数据边界、状态边界、事务边界和约束边界一次性写清楚，避免实现时出现理解分歧。后端技术栈固定为 Java 17 + Spring Boot 2.7.x，采用 Controller → Service → Repository 三层架构；前后端通过 HTTP GET/POST 传输 JSON；数据库使用 MySQL，且必须严格遵循既定 ER 图，不新增与核心业务无关的表，不改动既有实体关系。

系统中最重要的事实是：航段 FlightSegment 是最小可售区间，一个航班内的可售区间是预先拆分好的。订单 TicketSale 只对应一个航段，一个订单只买一个区间；如果要改签，则采用“上一张票 → 新票”的链式追踪方式，通过 OriginalTicketId 指向改签前一张票。库存字段位于 FlightSegment 中，分为头等舱余票与经济舱余票，购票时必须用数据库行级锁控制扣减，支付超时后统一由定时任务回补库存。支付逻辑为模拟支付：用户点击支付即返回成功结果，但后端仍必须完成状态流转、库存确认、积分发放、会员升级等数据库动作。

以下内容分为两份：第一部分是后端与前端的对接文档，第二部分是后端与 MySQL 数据库队友的对接文档。两部分都围绕同一套业务规则展开，但关注点不同。前端对接重点是页面、接口、参数、返回值、错误提示与状态展示；数据库对接重点是表结构、字段约束、索引、事务、锁、种子数据与状态枚举。

第一部分：后端与前端对接文档

1. 对接原则与通信约定

前后端必须采用 JSON 作为唯一数据交换格式，所有请求和响应都要统一返回 JSON。前端不直接拼接数据库字段，也不直接参与业务判定；前端只负责收集输入、展示结果、根据状态码和业务字段渲染页面。后端负责所有核心校验，包括登录校验、角色校验、字段合法性校验、库存校验、时间窗校验、权限校验与订单状态流转。

接口风格统一按 REST 思路组织：查询类接口优先使用 GET，请求参数放在 URL Query String 中；新增、修改、删除、登录、退票、改签、支付、注册等会改变系统状态的操作统一使用 POST。之所以不把所有接口都做成 GET，是因为课程设计演示时老师会重点观察你们是否知道“查询”和“有副作用操作”的区别。前端只需严格按接口文档调用，不要在页面层自行模拟库存变化或会员升级。

前后端联调时，前端必须把字段名原样传给后端，禁止自行改名、缩写或 camelCase 乱转。我们已经约定 JSON 字段名必须与数据库字段名严格对齐，因此前端提交的键名应与后端 DTO 以及数据库字段尽量保持一致，例如 `userId`、`flightId`、`segmentId`、`ticketStatus`、`mealId`、`priceAmount`、`paymentAmount`、`originalTicketId`、`idNumberDigest` 等。对于后端返回值，也应保持同样字段命名习惯，减少映射错误。

统一响应格式建议如下：`code` 表示业务状态码，`message` 表示提示信息，`data` 表示业务数据，`timestamp` 表示响应时间。前端只根据 `code` 判断成功或失败，不要根据 `message` 做逻辑判断。建议约定 `code=0` 为成功，`code=400xx` 为参数错误，`code=401xx` 为未登录或无权限，`code=404xx` 为资源不存在，`code=409xx` 为业务冲突，`code=500xx` 为系统异常。这样页面提示和调试日志都比较清晰。

2. 登录、注册与角色控制

系统支持两个角色：普通乘客与系统管理员。普通乘客可以注册、登录、查询航班、下单、支付、查看订单、退票、改签、选择餐食；系统管理员可以维护城市、机场、飞机、航班、航段、餐食选项，并查看所有用户和所有订单，但不能修改用户积分，也不能越权替用户购票。

登录接口建议为 `POST /api/auth/login`。请求体包含 `loginAccount` 与 `password`，其中 `loginAccount` 可以是登录账号或手机号/邮箱中的某一种，具体以实现为准，但对接时必须统一一种主登录字段，不能多套逻辑混用。后端登录成功后返回用户基本信息、角色信息、会员等级、积分值与一个前端可缓存的登录态标识。若你们最终采用 Session，则前端只需携带 Cookie；若采用 Token，则响应中返回 `token`，前端后续请求放入请求头。为了课程设计稳定，建议优先采用 Session 或简化版 Token，不要引入复杂的 OAuth 机制。

注册接口建议为 `POST /api/auth/register`。请求体至少包含 `loginAccount`、`password`、`userName`、`phoneNumber`、`email`、`idNumberDigest` 对应的明文身份证号输入字段。注意：身份证号在前端页面上应以明文输入，但后端落库前必须做不可逆摘要处理；前端不要自行加密。注册成功后，后端自动为新用户设置默认角色为普通乘客、默认会员等级为普通、默认积分为 0、默认创建时间与更新时间为当前时间。若账号、手机号、邮箱或身份证摘要重复，则返回明确的重复错误提示。

前端页面在登录后必须根据 `userType` 控制菜单显示。管理员登录后显示管理员菜单，普通乘客登录后显示乘客菜单。不要依赖前端隐藏按钮来实现权限，真正权限控制仍在后端拦截层和接口层完成。前端只是展示层，不能作为安全边界。

3. 乘客端核心功能对接

3.1 航班查询

航班查询是乘客端最重要的入口。建议前端页面提供出发城市、到达城市、出发日期起止、舱位类型四项筛选条件，其中舱位类型至少支持经济舱和头等舱。查询接口建议为 `GET /api/flight/search`。请求参数包括 `departureCityId` 或 `departureAirportCode`、`arrivalCityId` 或 `arrivalAirportCode`、`flightDateStart`、`flightDateEnd`、`cabinClass`。具体究竟按城市查还是按机场查，后端可兼容，但前端页面建议优先使用城市下拉框，因为课程展示更直观。

返回结果应包含航班基础信息、航段信息、票价、余票、起降机场、起降城市、飞行日期、航班状态、飞机型号、航段编号以及是否可售。若该航班当前已停用，则前端不得展示为可预订状态。若某个舱位余票为 0，前端可以显示“售罄”，但仍可保留在查询结果中，方便老师看到库存变化。

3.2 下单与预订

机票预订建议拆成两个步骤：先选择航班与航段，再进入填写乘客信息与餐食选择页面。下单接口建议为 `POST /api/ticket/create`。请求体必须包括 `userId`、`flightId`、`segmentId`、`cabinClass`、`passengerName`、`passengerIdNumber`、`mealId`。如果是登录用户本人下单，可以预填用户名与身份证信息；如果允许代他人购买，也可以手工填写乘客姓名与身份证。后端收到请求后，必须做以下校验：航班是否存在且可售、航段是否存在且属于该航班、舱位是否匹配、余票是否足够、乘客身份证是否重复占票、当前用户是否有权限下单。

下单成功后，后端返回创建好的订单信息，订单状态应为“待支付”，并记录支付截止时间 `expiredAt`。前端页面必须立即展示“15 分钟内完成支付”的提示，同时显示倒计时。前端只负责倒计时显示，不负责决定是否过期；真正过期判定由后端定时任务和支付接口双重校验。

3.3 支付

支付接口建议为 `POST /api/ticket/pay`。请求体只需包含 `ticketId`。由于系统是模拟支付，前端点击支付按钮后无需接入任何第三方支付 SDK，直接调用后端支付接口即可。后端在支付成功时必须完成：将订单状态改为已支付、写入 `paidAt`、写入 `issuedAt`、确认库存扣减、累计积分、必要时触发会员升级。如果订单已经过期，则支付接口必须返回失败，提示用户订单已超时。

前端在支付成功后应自动刷新订单详情与列表，展示最新状态、支付金额、积分变化和会员等级变化。若用户当前订单是会员升级前的最后一单，前端应在支付完成后立即刷新用户信息，确保“下一单九折”这一演示能顺利发生。

3.4 订单管理

订单列表接口建议为 `GET /api/ticket/my`，请求参数可包含 `userId`、`ticketStatus`、`pageNum`、`pageSize`。列表中至少展示订单号、航班号、航段、乘客姓名、票种、下单时间、支付时间、出票时间、订单状态、票面金额、实付金额、是否可退、是否可改签。订单详情接口建议为 `GET /api/ticket/detail`，请求参数为 `ticketId`。详情页要能穿透显示关联航班、航段、用户、餐食、改签来源票与退票时间等信息。

退票接口建议为 `POST /api/ticket/refund`。请求体包含 `ticketId`。系统规则已经确定为“任何时间都可退票”，因此前端无需展示 24 小时限制提示。后端收到退票请求后，若订单状态允许退票，则将票务状态改为退款成功或已退款，并回补对应航段余票，同时视需要回滚积分。前端在提交退票后应刷新订单状态与库存提示。

改签接口建议拆为两个步骤。第一步是改签发起，接口为 `POST /api/ticket/change/apply`，请求体包含 `ticketId` 与 `targetSegmentId` 或新的航班航段组合。第二步是改签支付，接口为 `POST /api/ticket/change/pay`，请求体包含改签单号或新票号。若新票价格高于原票，系统会生成差价支付金额；若差价为 0，则可直接确认成功。改签必须支持跨航班。改签成功后，旧票状态应变为改签成功或已作废，新票应作为一张新票存在，并通过 `originalTicketId` 追踪来源。

3.5 餐食选择

餐食选项固定为普通餐、清真餐、素食餐三种。餐食列表接口建议为 `GET /api/meal/list`，返回所有可用餐食及其状态。下单时前端将 `mealId` 一并提交。餐食是附属业务，但在演示时能体现系统完整性，因此前端页面应在购票页清晰展示这三个选项，并在订单详情中回显已选餐食。

4. 管理员端核心功能对接

管理员端的目标是稳定演示航班、航段、飞机、城市、机场等主数据的增删改查，以及让老师看到标准数据库业务闭环。管理员不负责用户积分修改，不负责越权操作订单，不负责直接调整库存。

4.1 城市与机场管理

城市管理接口建议包括城市新增、城市列表、城市修改、城市停用。机场管理接口建议包括机场新增、机场列表、机场修改、机场停用。城市表和机场表原则上可采用逻辑删除或停用，不做物理删除。机场新增时需要关联所属城市，且必须保证机场编码唯一。城市和机场数据可包含国内与境外样例，只要能支撑演示查询即可。

4.2 飞机管理

飞机管理接口建议包括飞机新增、飞机列表、飞机修改、飞机停用。飞机实体用于支撑航班与座位数量字段，管理员可维护飞机注册号、型号、制造商、头等舱总座位数、经济舱总座位数、状态和备注。若飞机已被航班引用，不建议物理删除，只允许停用或修改非关键属性。

4.3 航班管理

航班管理是管理员最重要的演示模块。航班列表接口建议为 `GET /api/admin/flight/list`，支持按航班号、日期、状态、出发地、到达地筛选。新增航班接口建议为 `POST /api/admin/flight/add`，修改航班接口建议为 `POST /api/admin/flight/update`，停用航班接口建议为 `POST /api/admin/flight/disable`。航班删除接口不建议做物理删除；如果系统内已有票务记录，则必须禁止删除，只能停用。

前端在管理员页面上应能完成“新增 → 查询验证 → 修改 → 再次查询 → 停用 → 最终确认”的完整演示。每一步都应该有明确的返回提示与列表刷新。若管理员修改了航班状态，前端应同步更新相关展示，避免出现页面与数据库不一致的情况。

4.4 航段管理

航段是可售区间的核心，管理员需要能够维护航段信息。接口建议包括新增航段、查询航段、修改航段、停用航段。航段字段至少包括所属航班、起点站序号、终点站序号、起始机场、终点机场、计划起飞时间、计划到达时间、头等舱剩余座位、经济舱剩余座位、头等舱价格、经济舱价格、状态与备注。由于你们的 ER 图已经确定了航段是最小库存单位，因此管理员不能只改区间名称而不改库存字段，也不能绕过航段直接改订单库存。

5. 前端页面结构建议

前端页面可以分为登录注册页、乘客端首页、航班查询页、下单页、订单列表页、订单详情页、管理员首页、城市管理页、机场管理页、飞机管理页、航班管理页、航段管理页、用户列表页、餐食管理页。页面之间最好采用统一的顶部导航加左侧菜单或顶部标签页结构，保证演示时切换简单。

乘客端最重要的是流程连续性：先登录，再查票，再下单，再支付，再查看订单，再退票或改签。管理员端最重要的是操作可视化：先新增，再查询，再修改，再再次查询，最后停用或删除验证。前端页面不需要追求商业级复杂效果，但必须做到表单清晰、表格密度合理、状态提示明确、按钮命名准确。

6. 前端联调注意事项

前端必须处理好以下四类返回：成功、参数错误、权限不足、业务冲突。成功时刷新列表和详情，参数错误时高亮表单字段，权限不足时跳回登录页，业务冲突时提示余票不足、订单已过期、航班已停用、改签差价未支付等信息。

前端不要缓存会变化的库存和积分作为最终真值，页面缓存只能用来展示，真正数据应以接口返回为准。尤其是支付、退票、改签后，必须重新拉取订单详情与用户信息，否则演示时很容易出现显示不一致。

第二部分：后端与 MySQL 数据库对接文档

1. 数据库对接原则

数据库设计必须严格遵循既定 ER 图，不新增与业务无关的实体，不破坏既有关系，不把航段库存拆散到别的表里。后端与数据库之间的关系不是“前端表单直接写表”，而是“Controller 接收请求 → Service 做业务校验 → Repository 执行事务性数据库操作”。任何涉及库存变化、积分变化、订单状态变化、会员升级变化的操作，都必须在事务内完成，不能分散成多个无保护的独立 SQL。

数据库层面的第一原则是可追溯。所有订单、改签、退票、支付、积分变化都应能通过主表和外键链条追踪到来源。第二原则是可控并发。购票和支付超时释放库存必须使用行级锁或等价机制控制超卖。第三原则是可演示。课程设计不追求极端高并发，但要保证逻辑正确、字段清晰、状态完整、查询稳定。

2. 表级对接说明

2.1 用户表 User

User 表存储登录与身份信息。后端需要使用该表实现登录、注册、角色控制、积分升级与会员等级显示。字段中 PasswordHash 必须存储 BCrypt 哈希结果，IdNumberDigest 必须存储不可逆摘要，不能保存明文身份证

号。 `UserType` 区分普通乘客与管理员， `MemberLevel` 区分普通与 VIP。 `Points` 记录积分， `CreatedAt` 和 `UpdatedAt` 用于演示用户新增与信息变更。

数据库约束方面， `LoginAccount`、`IdNumberDigest`、`PhoneNumber`、`Email` 建议分别建立唯一约束或至少为高选择性字段建立唯一索引，避免重复注册。若你们最终只想保留一个主登录字段，也可以将其他字段设为普通字段，但从演示和完整性角度看，账号、身份证摘要和手机号至少应避免重复。

2.2 城市表 City

City 表是最基础的地理维度。城市编码、城市名称、国家信息应保持稳定。机场表通过 `CityId` 外键关联城市。后端查询航班时，如果前端用城市作为筛选条件，则需要先通过 City 再联到 Airport 继续查 Flight。若采用机场编码直接筛选，则城市字段主要用于页面展示。

2.3 机场表 Airport

Airport 表存储机场编码、机场名称、所属城市、是否国际机场等信息。机场编码应唯一，且必须与城市关联。机场逻辑上不建议物理删除，因为航班历史与订单历史都可能引用机场信息。停用机场后，前端查询页面应不再展示该机场为可选项，但历史订单和历史航班仍应可见。

2.4 飞机表 Aircraft

Aircraft 表负责承载飞机注册号、型号、制造商、座位总数、状态与备注。 `AircraftRegNo` 为主键或唯一键，必须唯一。飞机与航班是一对多关系，航班通过 `AircraftRegNo` 外键引用飞机。若飞机被航班引用，后端应禁止物理删除，只允许停用或修改非关键字段。

2.5 航班表 Flight

Flight 表保存航班级别信息，例如航班号、航班日期、飞机注册号、出发机场代码、到达机场代码、航班状态、备注。根据你们的 ER 图与确认规则，Flight 不存计划/实际起降时间，也不存延误分钟，相关时间由 FlightSegment 推导或直接由区间记录承载。航班一旦存在票务记录，就禁止物理删除，只能停用。后端新增航班时需要先检查飞机与机场是否存在，避免外键失败。

2.6 航段表 FlightSegment

FlightSegment 是整个系统最关键的库存表，也是票务购买的直接目标。字段至少包括段 ID、所属航班、起点站序号、终点站序号、起始机场代码、终点机场代码、计划起飞时间、计划到达时间、实际起飞时间、实际到达时间、延误分钟数、延误原因、头等舱剩余座位、经济舱剩余座位、头等舱价格、经济舱价格、备注。虽然你们之前讨论过“航班级时间可由航段推导”，但在当前明确版本里，时间字段仍以 ER 图为准，不在别的表重复存储。

后端在购票时只锁这一张表中的目标航段记录。由于你们确认“每一个可售区间单独设立独立库存”，所以 A→B、B→C、A→C 之间的库存互不联动。这样做实现简单，演示稳定，符合课程设计目标。库存扣减必须在事务内完成，且在执行扣减前先对目标 `SegmentId` 记录加行锁，避免多人同时购买最后一张票。

2.7 TicketSale 表

TicketSale 是订单与机票事实表。它记录 TicketId、OrderNo、UserId、FlightId、SegmentId、CabinClass、TicketStatus、PassengerName、PassengerIdNumberDigest、PriceAmount、PaymentAmount、OriginalTicketId、ChangeReason、BookedAt、PaidAt、IssuedAt、ChangedAt、RefundedAt、ExpiredAt、Remark 等字段。此表的状态流转是整个系统最容易出错的地方，因此后端必须在设计上把所有状态一次性写清楚。

你们当前已经明确的状态包括：待支付、已支付、已过期、已退款、退款成功、改签成功、停用或其他必要辅助态。建议后端将状态流分成“订单过程态”和“结果态”两层理解：待支付、已支付、已过期属于过程态；退款成功、改签成功属于结果态。支付成功后应写入已支付；一旦出票完成，也可同时记录出票时间。若超时未支付，系统应标记为已过期并释放库存。若退票成功，则标记为退款成功或已退款。若改签完成，则旧票标记为改签成功或作废，新票作为新记录插入。

`OriginalTicketId` 的语义已经明确为“指向上一张票”，因此改签链是链式的，不回指最初票。后端在写入改签票时必须保留来源票号，以便详情页穿透展示。`PaymentAmount` 记录最终实付金额，`PriceAmount` 记录票面金额，二者可能因为 VIP 折扣而不同。数据库层建议保持两个字段都存在，这样老师在看表结构时能清楚区分“标价”和“成交价”。

2.8 MealOption 与 MealReservation 表

MealOption 维护普通餐、清真餐、素食餐三种餐食。字段包括 MealId、MealName、MealType、IsAvailable、Description。MealReservation 是 TicketSale 与 MealOption 之间的关联表，用来体现一张票对应一次餐食选择或多条关联关系。后端在下单时需要把餐食选项和订单一起写入事务，确保订单成功但餐食缺失的情况不会出现。

2.9 关系与外键

City → Airport 是一对多，Airport → Flight 是出发和到达两条外键关系，Flight → FlightSegment 是一对多，User → TicketSale 是一对多，Flight → TicketSale 是一对多，FlightSegment → TicketSale 是一对多，TicketSale → MealReservation 是一对多，MealOption → MealReservation 是一对多，TicketSale 自身通过 OriginalTicketId 形成自引用链。后端在写 Repository 层 SQL 时必须严格按照这些关系 JOIN，不要绕开外键直接堆拼字段。

3. 事务边界与并发控制

3.1 下单事务

下单事务必须包含：查询航段并加行锁、检查余票、扣减余票、插入 TicketSale 订单记录、插入餐食关联记录、设置支付截止时间 `expiredAt`。这几个步骤必须在同一个事务中执行。若任一步失败，整体回滚。最核心的 SQL 思路是先 `SELECT ... FOR UPDATE` 锁定对应 `FlightSegment` 行，再判断库存，成功后更新余票并插入订单。

3.2 支付事务

支付事务必须包含：检查订单是否仍为待支付、检查是否超过 `expiredAt`、将订单状态更新为已支付、写入 `paidAt` 与 `issuedAt`、发放积分、判断是否达到 VIP 阈值、必要时更新会员等级。支付是模拟支付，但数据库动作不能省略，因为演示时老师会查看订单状态变化。

3.3 超时释放事务

支付超时释放采用定时任务方案。后端每隔固定时间扫描待支付且已超时的订单，按订单关联的 `SegmentId` 回补库存，再将订单状态改为已过期并记录 `ExpiredAt`。这部分建议单独做一个定时服务，避免把超时逻辑散落在支付接口中。若用户在最后一刻点击支付，支付接口仍要再次检查超时，防止与定时任务竞态。

3.4 退票事务

退票事务应执行：验证订单状态可退、将订单状态改为退款成功或已退款、回补航段库存、回滚积分、必要时重算会员等级。由于你们已经放宽为“任何时间都可退票”，因此后端不再需要额外的起飞前限制校验，但仍应校验订单状态是否是已支付或已出票，避免重复退票。

3.5 改签事务

改签事务是“旧票退掉 + 新票生成 + 差价支付”的组合事务。若存在差价，则新改签单先进入待支付状态，用户在 15 分钟内完成补差价支付后，旧票正式失效，新票生效；若超时未支付，则新改签单过期，旧票保持原状态或按你们定义回滚。改签支持跨航班，因此后端必须允许目标航班与原航班不同，只要目标航段可售即可。改签成功后，旧票与新票之间要通过 `OriginalTicketId` 保持链式关联。

4. 字段约束与索引建议

数据库对接时，后端同学应要求数据库队友至少为以下字段建立约束：User 表的 `UserId` 主键、`LoginAccount` 唯一、`IdNumberDigest` 唯一、`PhoneNumber` 唯一或高选择性索引；City 表的 `CityId` 主键、`CityCode` 唯一；Airport 表的 `AirportCode` 主键、`AirportName` 非空、`CityId` 外键索引；Aircraft 表的 `AircraftRegNo` 主键；Flight 表的 `FlightId` 主键、`FlightNumber` 与 `FlightDate` 组合索引；FlightSegment 表的 `SegmentId` 主键、`FlightId` 外键索引、`OriginAirportCode` 与 `DestinationAirportCode` 索引、`OriginStopNo` 与 `DestinationStopNo` 组合索引；TicketSale 表的 `TicketId` 主键、`OrderNo` 唯一、`UserId`、`FlightId`、`SegmentId`、`OriginalTicketId` 的外键索引；MealOption 表的 `MealId` 主键、`MealName` 唯一；MealReservation 表的联合索引 `TicketId + MealId`。

`NOT NULL` 约束要尽量明确，尤其是用户登录字段、航班编号、航段编号、订单号、库存字段、状态字段、金额字段、时间字段都不应允许空值。金额字段建议使用 `DECIMAL(10,2)` 或更高精度，不要使用浮点类型。时间字段统一使用 `DATETIME`，不要混用字符串。

5. 种子数据与初始化要求

数据库队友需要准备可直接落库的完整初始化数据。最低要求是：至少 20 条航班数据，覆盖 5 到 6 个城市与 5 到 6 个机场，时间范围固定在 2026 年 7 月 1 日前后三天，即 2026-06-28 到 2026-07-04。为了保证演示时“任意选择一个出发地和一个目的地都能查到结果”，建议在种子数据中设计成多城市互通网状结构，而不是单一路线。

用户种子数据至少 3 个账号，其中 1 个管理员、2 个乘客。乘客中至少 1 个账号预置 900 积分，另一个普通积分较低，方便演示“再买一张票升级 VIP，再买一张票展示九折优惠”。餐食种子数据固定为普通餐、清真餐、素食餐三种。飞机数据建议至少 5 到 6 架，以支撑多个航班使用不同机型。航段数据要足够覆盖不同城市之间的直达区间与经停拆分后的区间，确保查询与下单都能演示到。

6. 数据库队友交付要求

数据库队友最终应交付以下内容：完整建表 SQL、主外键与索引 SQL、初始化数据 SQL、状态枚举说明、事务执行建议、并发控制说明、测试数据说明。后端实现时将直接依赖这些 SQL 和字段命名，因此数据库侧不得擅自改字段名、改字段类型、改主键策略或改外键关系。任何字段变更都必须先与后端确认，否则会直接导致 DTO 映射失败、接口报错或事务失效。

结束语

这份文档的核心目标，是让前端、后端、数据库三方在同一套字段、同一套状态、同一套业务规则下开发，不出现“页面能点但数据库写不进去”“数据库能存但接口返回对不上”“订单状态改了但前端看不出来”这类联调问题。只要三方严格按本文档执行，后端实现就可以稳定落地，演示时也能把查询、下单、支付、升级、退票、改签、停用、库存释放这些关键链路完整展示出来。